Unit - 3

CPU and microprogrammed control Unit

Data Transfer and manipulation instructions

→ It is of 3 types
→ these instructions are based on processor and (or) controller

1. Data transfer instructions
2. Data manipulation instructions.
3. Program control instructions.

1. Data transfer instructions :-

* Moves data from one location to other location
* location may be R-R (or) R-M (or) R-I/O.
* content will not be change

Typical Data Transfer Instructions

Name	Mnemonic
load	LD.
store	ST
Move	MOV
Exchange	XCH.
Input	IN
Output	OUT
Push	PUSH
Pop	POP

2. Data manipulation instructions :-

* these are used for performing operations on data present in R (or) M
* It is are of 3 types.
  1. Arithmetic 2. logical 3. shift

1. Arthamatic

Name	Opp code
Addition	ADD
subtraction	SUB
Addition with carry	ADDC
Subtraction with borrow	SUBB
multiply	MUL
Negate (2's	NEG
Increment	INC
Decrement	DEC

2. logical :-

* these are performed on string of bits or individu bits.

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive -OR	XOR
clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interupt	EI
Disable interupt	DI

3. Shift Instructions

Name	Mnemonics
logical shift right	SHR
logical shift left.	SHL
Arithmetic shift right.	ASHR
Rotate right	ROR
Rotate left.	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Example program :-

```
MOV R1, 2
INC R1
COM R1
SHR R1
```

3. Program control Instructions

Name	Mnemonic
Branch	BR
Jump.	JMP
skip	SKP
call	CALL
Return.	RET
compare (by subtraction)	CMP
Test (by ANDing)	TST
Reset	RST

* Conditional Branch Instructions

BZ	–	Branch if zero.
BNZ	–	Branch if nozero.
BC	–	Branch if carry
BNC	–	Branch if nocarry
BP	–	Branch if plus.
BM	–	Branch if minus.
BHI	–	Branch if higher
BHE	–	Branch if Equal.
BNE	–	Branch if not Equal
BGT	–	Branch if Greater than
BLT	–	Branch if lessthan.

* Addressing modes :-

→ Every program has instruction, each instruction undergoes f (fetch), D (Decod), E (Execute)

→ Each intructuctions has opcode, address, operan

→ Operands are Nothing but Data (values).

→ Operands can be given in Register or Memory.

Definition of Addressing mode :-

" It defines the way the operand is definedby the user".

Types of Addressing Modes :-

1. Immediate Addressing Mode
2. Register Addressing Mode.
3. Register Indirect Addressing mode.
4. Direct Addressing Mode.

5. Indirect Addressing Mode.
6. Implied Addressing Mode.
7. Relative Addressing Mode
8. Indexed Addressing mode.
9. Base Register Addressing Mode.
10. Autoincrement or Atuto decrement Addressing mode.

1. Immediate Addressing Mode:-

→ the operand is specified with in the instruction.
→ Operand itself is provided in the instruction rather than its address.

Ex :- MOV A, 15h (h = hexa) move immediate
MOV B, 5h A ← 15h
here 15h is the immediate operand.

2. Register Addressing Mode:-

→ Operand is present in registers.

Ex :-
MOV R_1, R_2 ⇒ value of R_2 is stored in R_1.
MOV A, B ⇒ value of B is moved to A.

3. Register Indirect Addressing mode:-

→ operand is present in memory location.
→ memory is denoted by '[ ]'

Ex :- MOV R_1, [5000H]
⇒ The operand at 5000 address is loaded into R_1.

4. Direct AM :-

→ the operand is present in the address given in instruction.
→ I = 0 also called as Immediate address.

Ex :- load Accumulator
LDA [4000]

STA [2000]

5. Indirect AM :-

→ Operand is present at Address of Address alg called as effective address when I = 1

Ex :- MOV R1, [[5000 H]]

6. Implied AM :-

→ opcode is a part of instruction.

→ also called as inherent addressing mo

Ex CMA → complement Accumulator.

CMC → complement carry flag

STC → set carry flag.

RSTC → Reset

Ex :- write a program with register value 5 initially & make that the register value '0'.

Ans MOV R1, 5

CLR R1

7. Indexed Addressing Modes:-

→ The opcode address is calculated by adding index register address + instruction address.

EA = Base address + index register

Q find the EA if base address is 5000H index address is 2000H

Ans EA = 5000 + 2000

= 7000 H

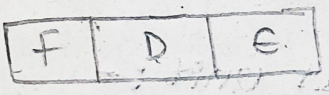
* RISC :-

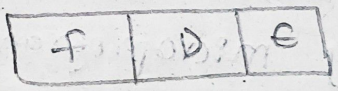
→ RISC ⇒ Reduced instruction set computer
→ Every processor or controller follows RISC or CISC.
→ Instruction set varies based on RISC or CISC.

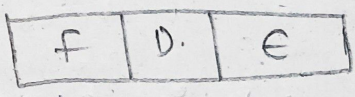
Characteristics of RISC :-

1. Simple instructions.
2. few addressing modes.
3. Instructions are executed in one ~~cycle~~ clk cycle.
4. It is based of software.
5. fixed size instructions.
6. Transistors are used for storing instructions.
7. It has 3 stage pipeling.
8. It has higher speed.

Consider 3 instructions,

1. 

2. 

3. 

Examples :-

→ 8086, 8051, 8085 follows RISC architecture.
→ ARM, ATMEL follows CISC architecture.

CISC :- Complex instruction set computer.

* CONTROL MEMORY :-

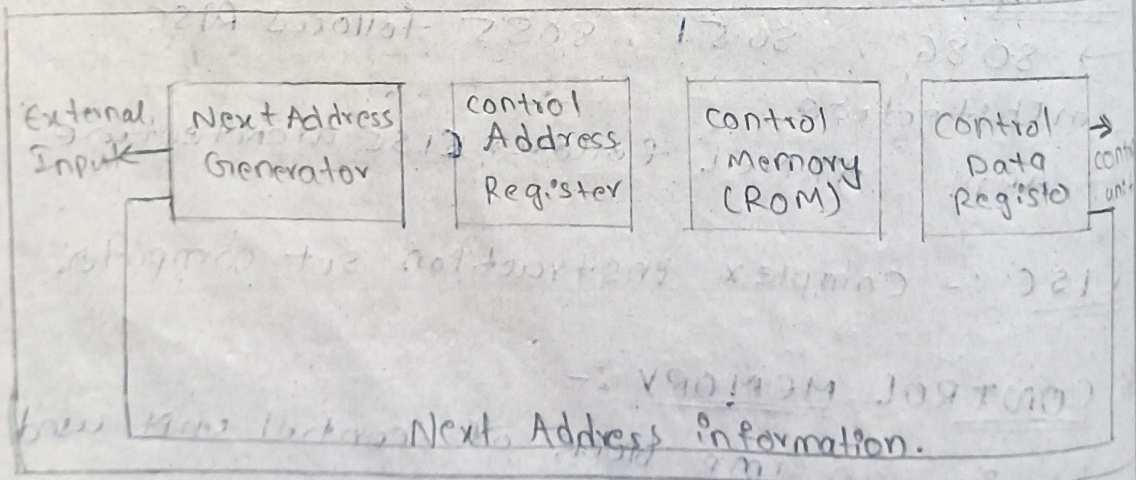
→ It is an imp block of control unit used to store micro operations.
→ Airthamatic, logical, shift are micro peration

→ It is made up of ROM.
→ It is Non volatile memory
→ only Read operation takes place
→ used by CPU to execute instructions in faster way. //

→ control unit is majorly responsible for producing control signals and timing signals for performing any task in the system. using ALU and memory units.

→ It is designed by 2 methods.
1. microprogrammed control unit.
2. Hardwired control unit.

1. Microprogrammed control unit :-
It is designed by microprogramme which is software.
→ Instruction set is CISC.



→ control Memory stores micro operations.
→ NAG acts as pc holds next address of micro instructions

Control Memory

→ control memory stores the operands & opcode to perform specific operation.

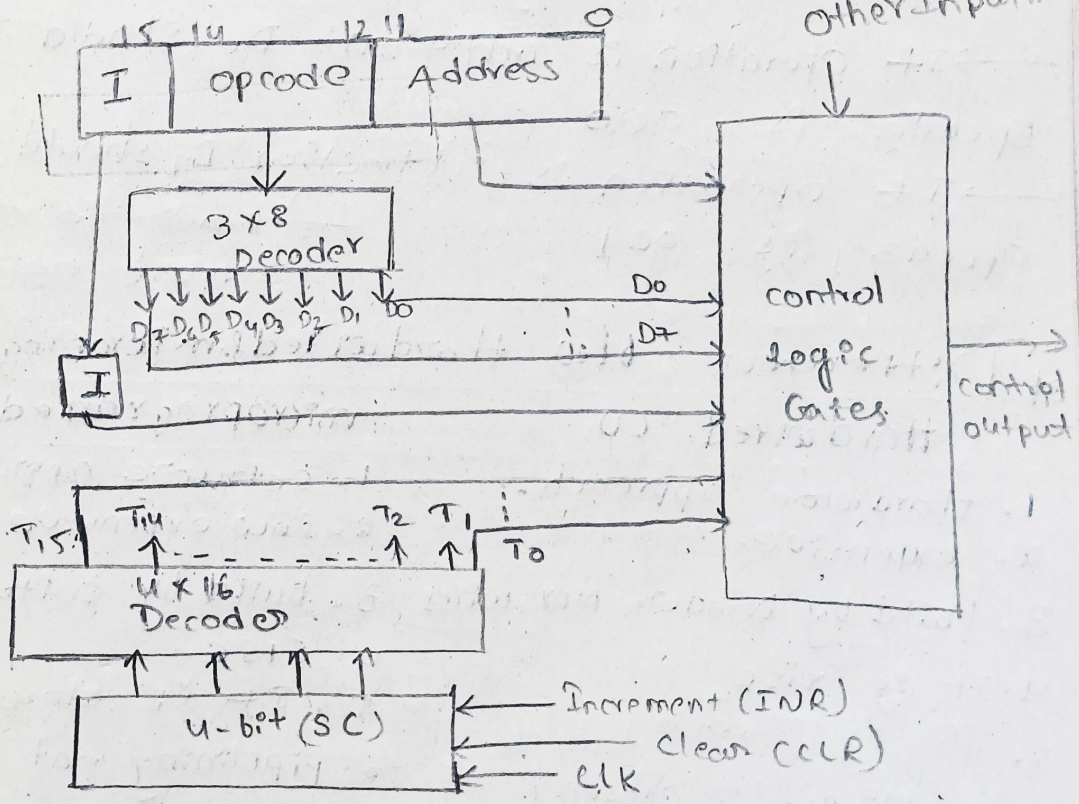
→ control unit responsible for successful operation.

→ CDR (control Data Register) holds data.

2. Hardwired control unit :-

It is a Hardware approach designed by Hardware devices (Decoders, Encoders, multiplexer, Demultiplexer, Cables etc).

→ The Operation is performed using control signals based on opcode with timing signal



→ Any operation will be performed by taking one output of 3 x 8 decoder ($D_0 - D_7$) combined with 1 output of timing signal ($T_0 - T_{15}$)

Example :- let us consider

Decoder o/p	opcode	Operand
D_0	000	ADD
D_1	001	SUB
D_2	010	MUL
D_3	011	DIV
D_4	100	SHR
D_5	101	AND
D_6	110	CLR
D_7	111	SHL

→ If Operation is addition D_0 should be activ. Opcode is 000

→ If operation is subtraction D_1 should be activ Opcode is 001

According to marks increse explination points

* Differences b/w Hardwired & microprogramed.

Hardwired CU	microprogramed CU
1. Hardware approach	1. software approach
2. expensive.	2. low expensive
3. build by Decoders, MUX, wires	3. build by software.
4. It is risc	4. CISC.
5. It is faster	5. It is slower compact Hardware
6. pipelining is present	6. pipelining not present.
7. Not easily to modified.	7. easily modified.
8. ROM is not used	8. ROM is used.

* Address sequencing :-

→ used for identifying next address to be executed.

→ NAG is responsible for identifying the next address

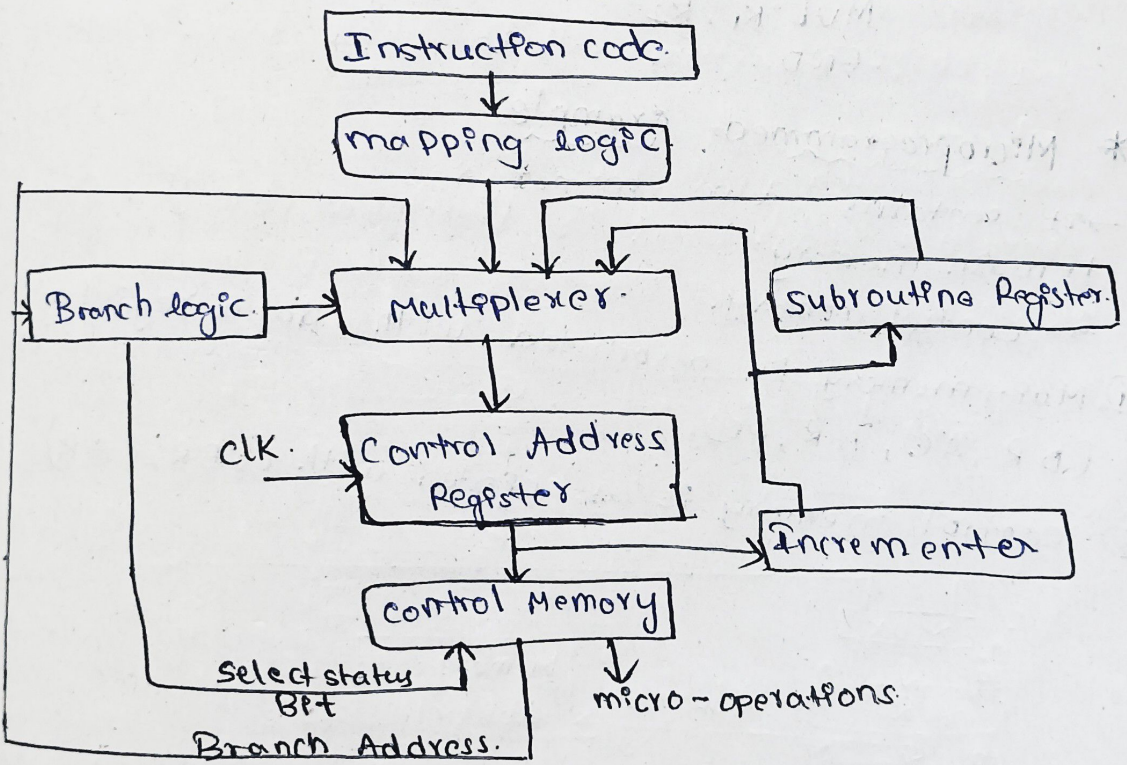
→ next address is executed based on program.

→ generally 4 ways are present for next address identification

1. Incrementing the CAR (control Address register)
2. checking the ~~subroute~~ Subroutine registes (call & Return function).
3. checking the condition and uncondition branches using branch logic
4. Mapping of computer instruction with microprogramed instruction.

→ Also called as microprogrammed sequencing

→ It is a part of microprogramed control Unit



Ex :- MOV R1, 5
(1) MOV R2, 5
ADD R1, R2
END.

The next instruction is executed without any loops

Ex :- 1. MOV R1, 6
(2) 2. DIV R, 2
3. CMP(R=0)
4. Printf(even)
5. Printf(odd) ×

→ The next address depends upon condition given in the program.

③

→ The next address depends upon call & return. Based on used programming.

Ex :-

```
MOV R1, 2
MOV R2, 4.
call.
{ ____
  ____
  ____
}
Return.
MUL R1, R2
END.
```

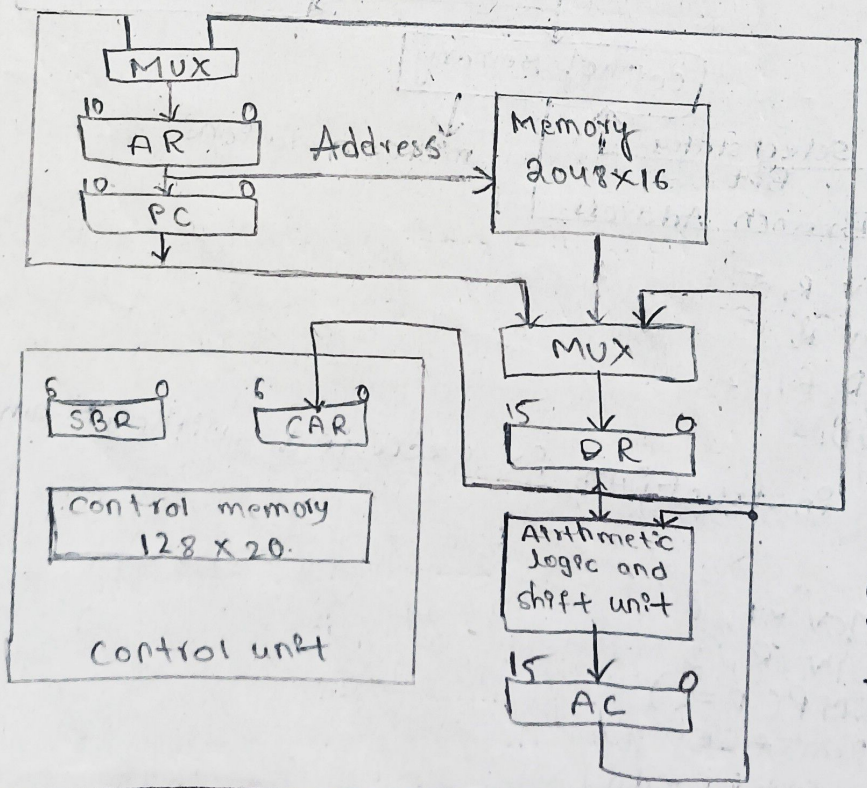
* Microprogrammed example

→ It contains two memorys

1. main memory.
2. control memory.

① Main memory is associated with data register (DR, AC, AR, PC).

② control memory is associated with (SBR, CAR)



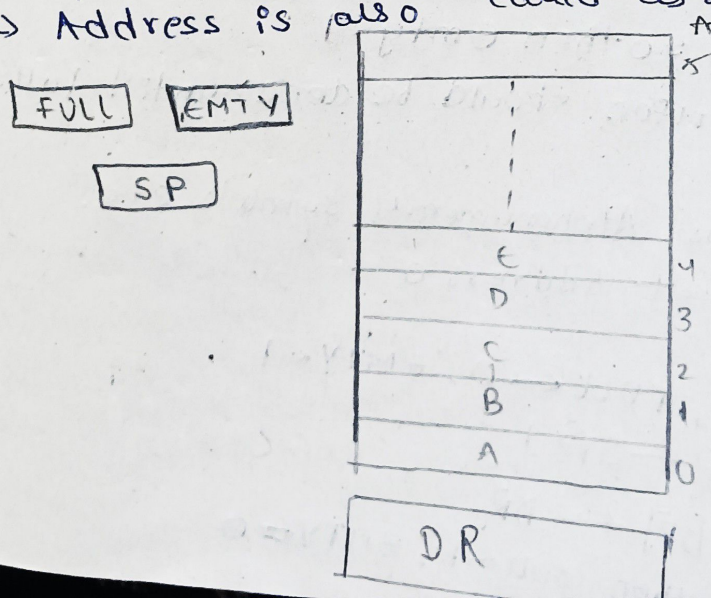
Computer configura

⟹ Main memory (MM) configuration is 2048×16 ($2^{11} \times 16$)
where 11 ⟶ Address lines
16 ⟶ Data lines

⟹ Control Memory (CM) configuration is 128×20
($2^7 \times 20$)
where 7 ⟶ Address lines. (CAR)
20-bit Data bus

* The data bus depends upon processor ~~and~~ or controller. MOX are used ~~for~~ performing single operation at a time.
* Operation is performed using AR, DR, PC, AC.
* CAR and SBR are responsible for performing for microoperation.
if SBR = 1 ⟶ then sub routine functions are present
if SBR = 0 ⟶ No call and return functions.
* The required data is sent from CM to MM.
* processor language is. ALP ⟶ Assembly language programming.

* stack organization
⟶ The principle of stack is first in last out
⟶ operations on stack is pop & push
⟶ push stands for insertion of data.
⟶ pop stands for deletion of data.
⟶ FULL, EMPTY ... are called flag register stores 1 bit of data.
⟶ SP ⟶ stack pointer holds the addres of the variable
⟶ Address is also called as location.

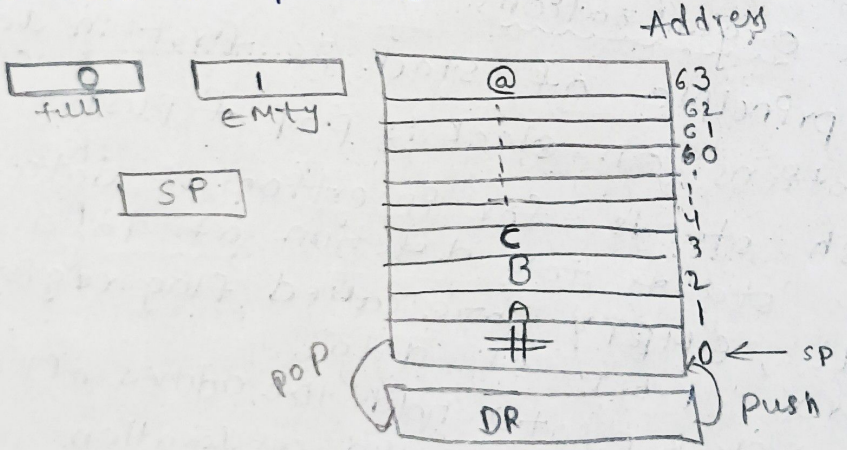


→ If full = 1 → stack is full
full = 0 → stack is not full (or) (stack is empty)

→ If EMTY = 0 → stack is not vacant (FULL)
EMTY = 1 → stack is vacant.

* Types of stack.
1. Register stack organisation.
2. Memory stack organisation.

① Register stack organisation
* DR is used for sending the data into stack (push) Recieving the Data from stack.
* Let us consider 64 bit register stack memory with address 0 to 63
* Condition :- Insertion will be done from address for push
1, 2, 3 ------ 63
Last insertion will be at '0'
Operation here used is increment (INC) or (INR)
Sp is used for pointing the address location.



→ Initially full = 0 then Emty = 1
→ the data insertion should be done until full = 1 or Emty = 0.
→ At DR contains Alphanumeraly symbols etc.
→ Initially sp is at Address 0.

Push operation
SP ← 0, FULL ← 0, EMTY = 1
SP ← SP + 1
M[SP] ← DR
If SP = 0, then FULL ← 1, EMTY ← 0

Pop Operation :-

The deletion starts from location '0' and continues from 63, 62, ---- 1

→ full = 1, EMTY = 0, sp at '0'

→ Decrement operation should be performed.

Program :-

SP ← 0, full ← 1, EMTY = 0

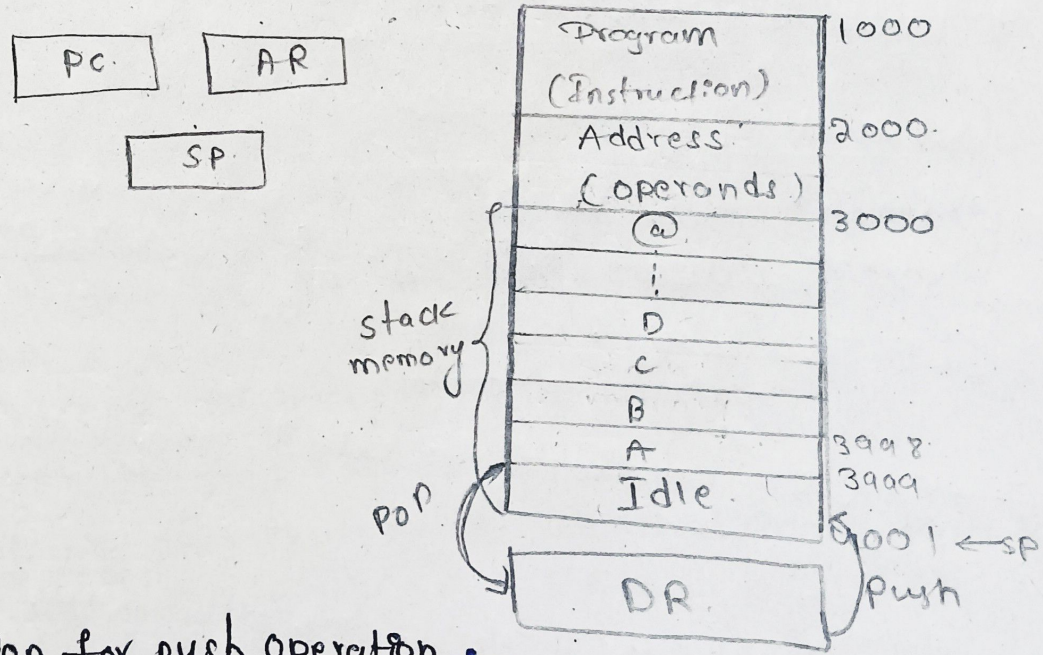
DR ← M[SP]

SP ← SP - 1

If SP = 1, then full ← 0, EMTY ← 1.

② Memory stack organisation :-

* Here stack is divided into 3 parts:
  1. program 2. Instruction 3. stack memory.
* PC, AR, SP are used
* let the starting address for program be 1000.
  ending address 2000
* starting address for operands 2000
* ending address 3000.
* starting address for memory stack 3000.
* ending address be 4001



Condition for push operation :-

* Insertion starts from 4000, 3999, 3998 ---- 3000
* Operation is decrement.
* The insertion operation is performed upto SP greater than or equal to starting address of stack.

Program for push operation

SP ← 4001
SP ← SP - 1
M[SP] ← DR
If (SP ≥ 3000) then FULL ← 1, EMTY ← 0

Condition for pop operation:-

→ It is performed upto SP less than ending address of stack.

→ let SP be at 3001. The operation performed is increment

Program for pop operation:-

SP ← 3001
DR ← M[SP]
SP ← SP+1
If (SP ≤ 4001) then full ← 0, EMTY ← 1